

Project 1: Motion detection

实验报告

一. 小组成员及分工: (各 25%)

何衍宁 Task1+报告

夏天 Task1+报告

周晓俊 Task2+PPT

张安泰 Task2+PPT

二. 背景介绍:

(一) 多路径传播

多路径传播指的是无线电信号通过两条或多条路径同时到达接收天线。由于环境中存在建筑物、山体、地面、天际线等障碍物，信号在传播过程中可能会发生反射、折射或绕射，从而形成多条不同的传播路径。这些路径之间的传播时间、距离、相位等特性可能存在差异，导致信号到达接收点时，既有直接路径的信号，又有反射路径、折射路径或绕射路径的信号。信号反射和衍射的存在，使得接收机接收到的信号不再是单一的直射波，而是多个路径上信号的叠加。由于信号在传播过程中经历了不同的传播路径，它们的强度和到达时间可能有所不同，这种信号的叠加可能会出现相位差异，从而导致干涉现象。干涉现象可能是建设性的，也可能是破坏性的。建设性干涉会增强信号强度，而破坏性干涉则可能导致信号衰减，产生所谓的"多径衰落"。

(二) 雷达

该装置可以通过分析物体表面反射的高频无线电波，以确定物体的位置、速度和其他运动状态，用于精确探测目标物体并获取其动态信息。

主动雷达: 主动雷达系统通过发射高频电磁波（如微波或无线电波）向目标物体发送信号。这些信号在碰到目标物体后会被反射回来，接收机捕捉到这些反射信号并对其进行处理。

无源雷达: 与主动雷达不同，其不依赖于自身的信号发射器，而是利用外部信号源（如通信基站、电视广播发射塔、卫星等）作为信号发射源。在这种系统中，基站或其他外部发射机提供参考信号，而雷达设备则充当接收机，接收来自不同目标物体的散射信号。由于目标物体的反射表面会改变信号的传播路径，这些反射信号会与参考信号之间存在一定的时延和频率差。

(三) 多普勒频移

多普勒频移是由信号源与观测器（或接收器）之间的相对运动引起的一种现象。当信号源和接收器相对运动时，接收到的信号频率会发生变化。这种频率的变化是由于波源发出的波的传播与接收机的运动状态之间相互作用

所造成的。具体来说，当信号源和接收器之间存在相对运动时，接收器感知到的信号频率会因为运动而发生偏移。当信号源和接收器靠近时，波的波长会变短，从而使接收器接收到的频率增加，这种现象被称为“蓝移”或正频移。当信号源和接收器远离时，波的波长会变长，接收到的频率减少，这种现象被称为“红移”或负频移。波源振动后会发出一个波，而接收机由于相对运动而接收到不同的频率。

$$f' = \left(\frac{v \pm v_0}{v \mp v_s} \right) f$$

公式 1 频移公式

三. 项目目标

利用被动雷达对目标监测得到的二十组数据，分析并得到被测目标的距离与速度。利用被动雷达对目标进行监测时，通常通过接收外部信号源（如基站、广播塔或卫星等）发射的电磁波信号，并分析目标反射回来的信号来获取目标的动态信息。

四. 项目内容

(一) Task1

1.任务描述：

对 20 组信号分别进行频移，低通滤波处理，并分别画出监测信号与参考信号的原始时域频域图、频移后的时域频域图、低通滤波后时域频域图。

2. 实验过程：

- (1) 首先将原始数据导入 Matlab，并画出参考信号，监测信号的时域，频域图。

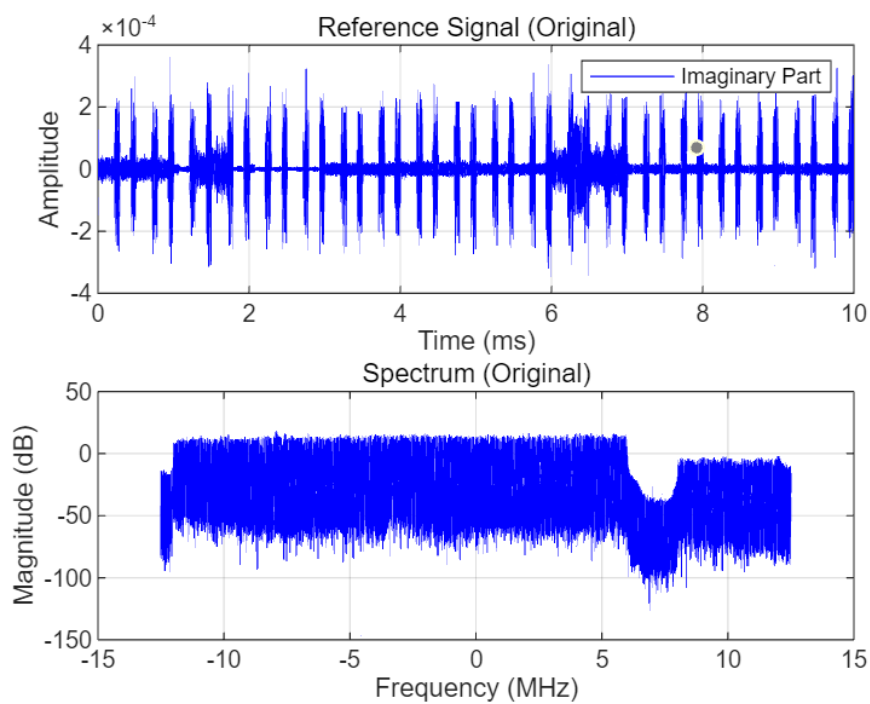


图1 参考信号的时域图局部图（虚部）+完整频域图

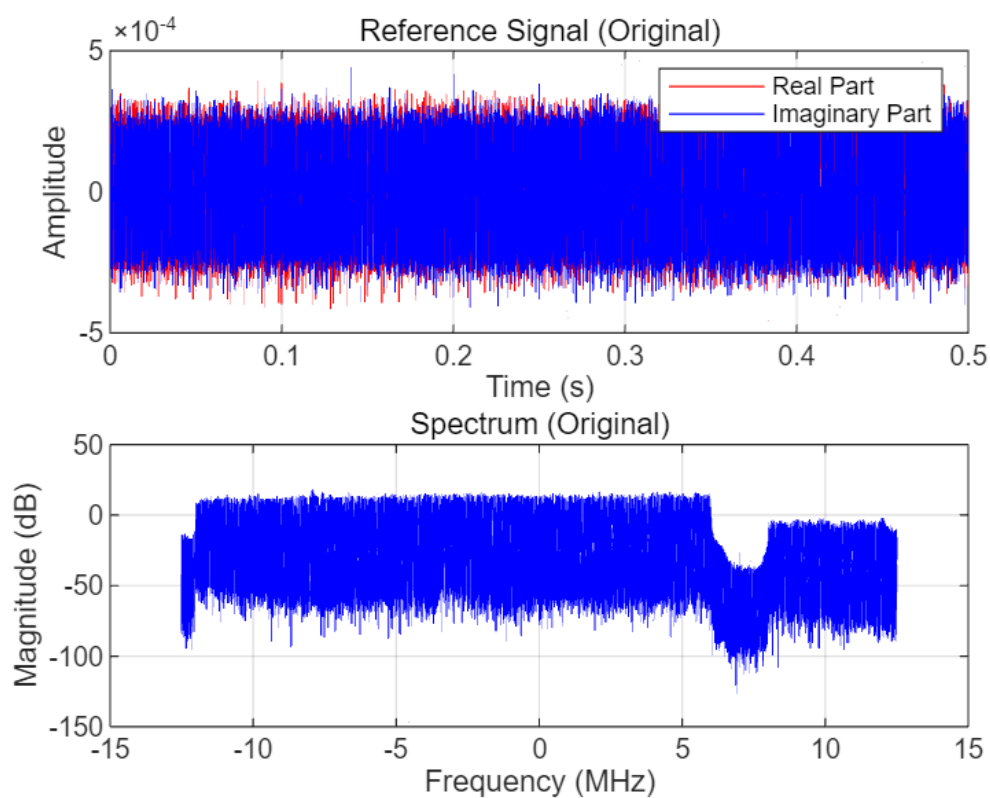


图2 参考信号的完整时域图+完整频域图

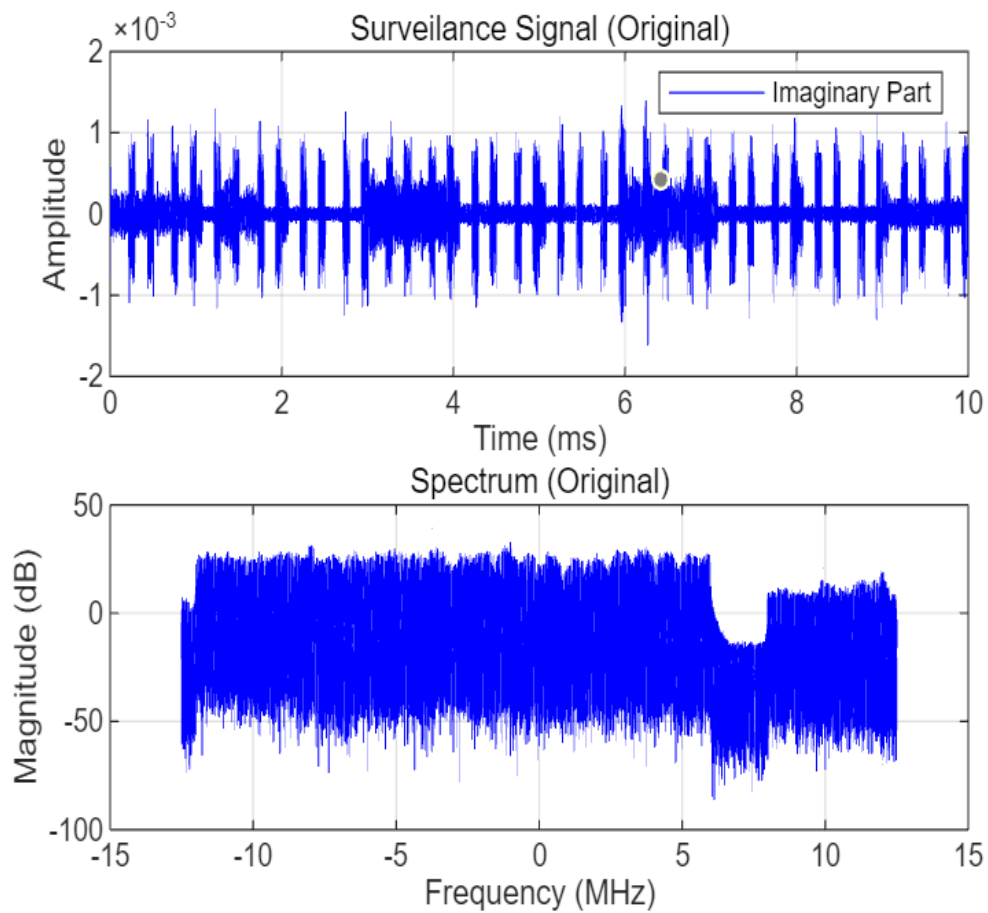


图3 监测信号的时域图局部图（虚部）+完整频域图

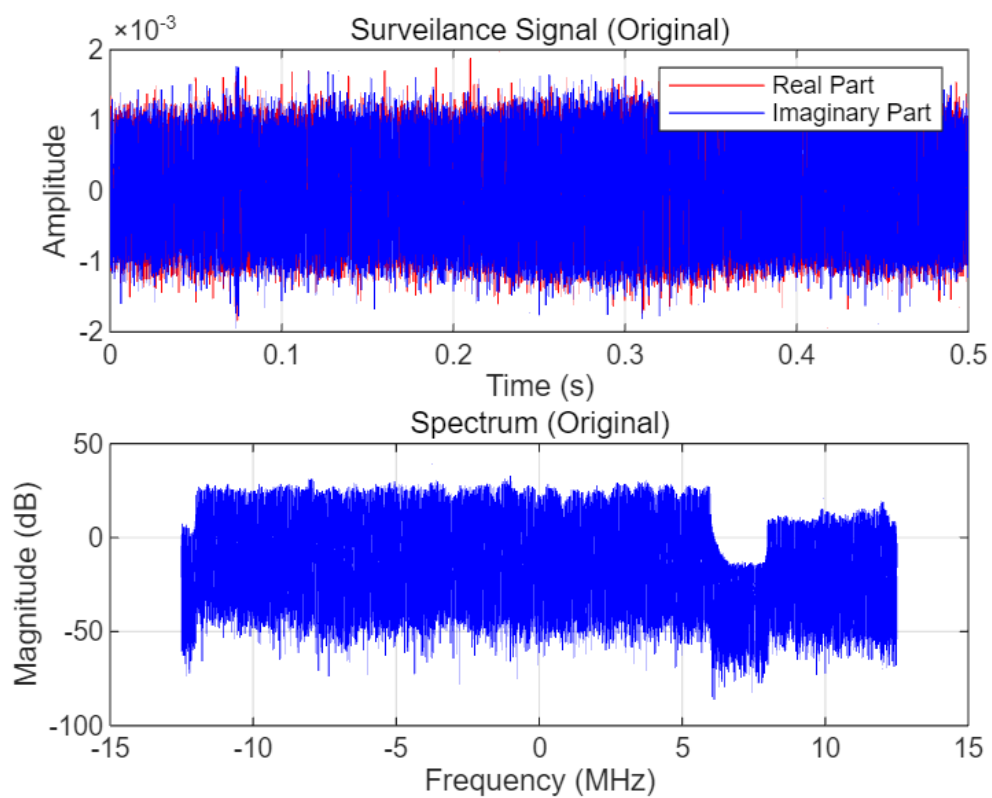


图4 监测信号的完整时域图+完整频域图

(2) 对两个信号分别做频移处理，并画出时域频域图。

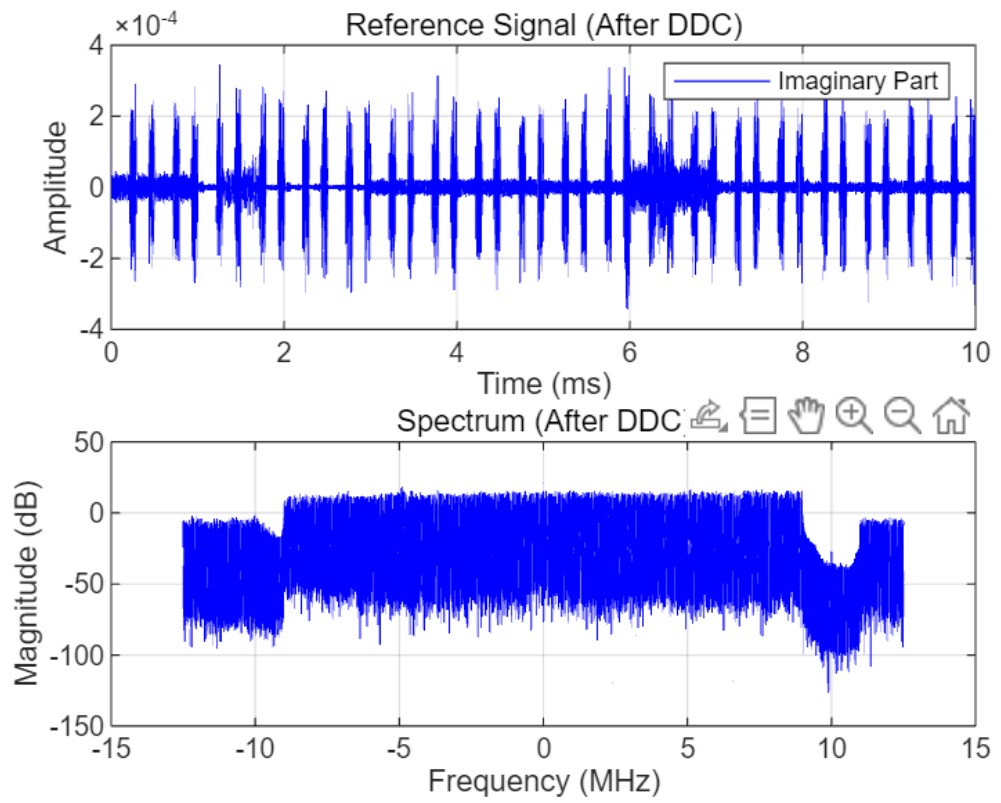


图 5 参考信号时域图局部图（虚部）+完整频域图(After DDC)

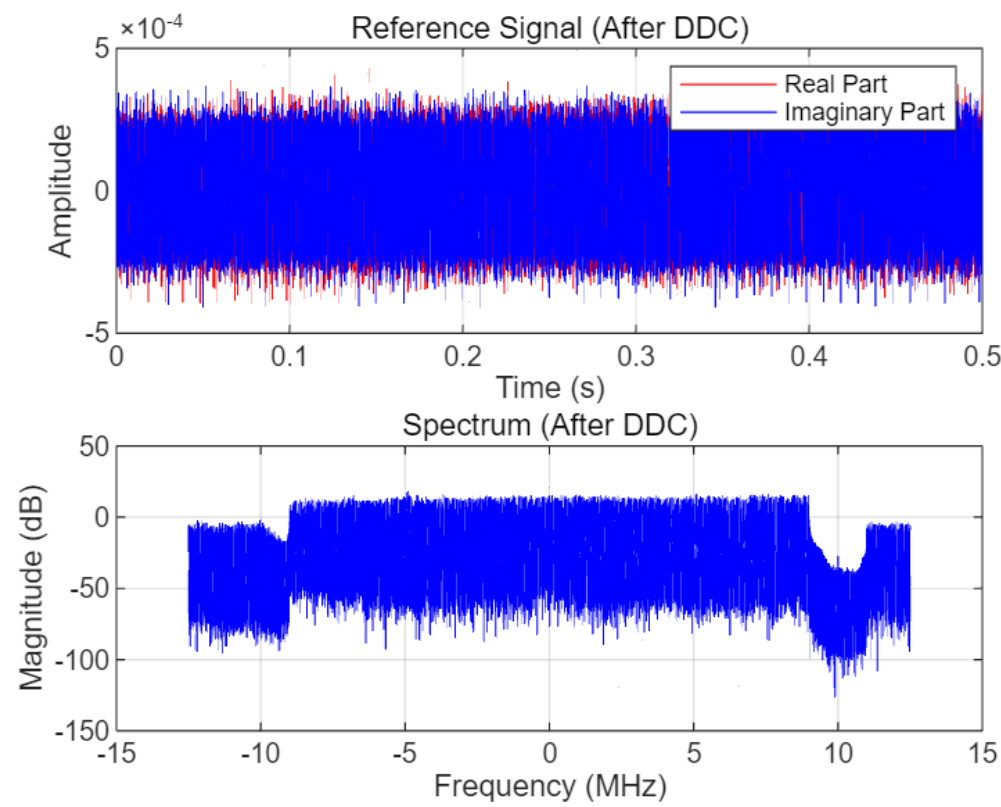


图 6 参考信号完整时域图+完整频域图(After DDC)

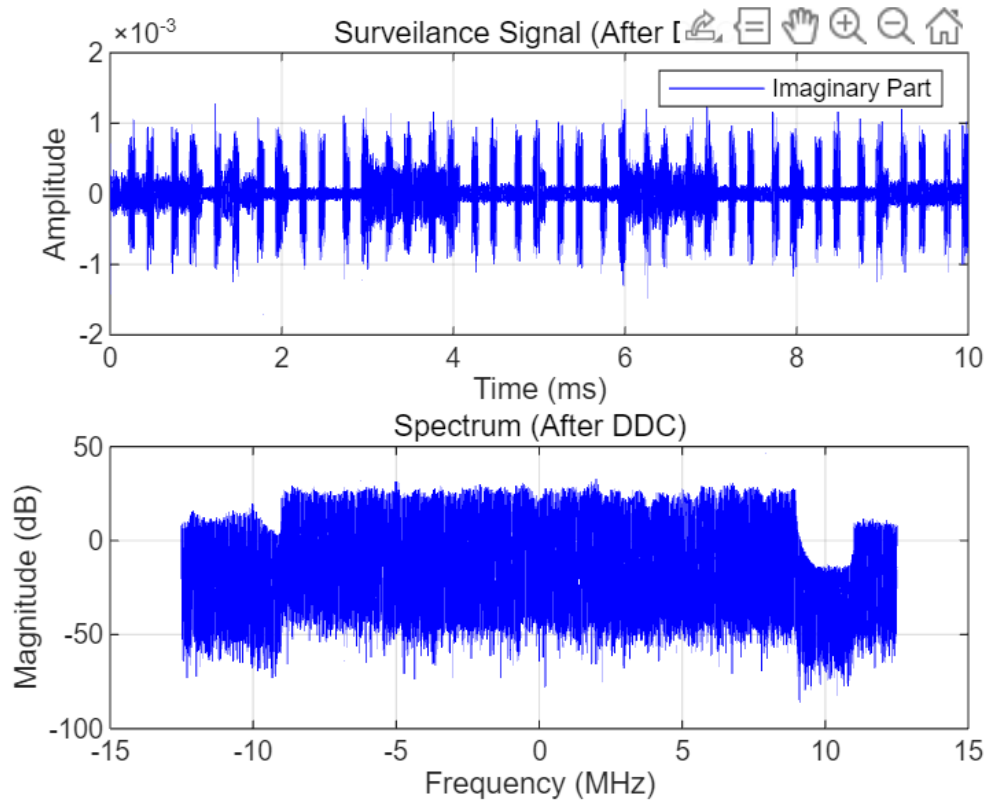


图 7 监测信号时域图局部图（虚部）+完整频域图(After DDC)

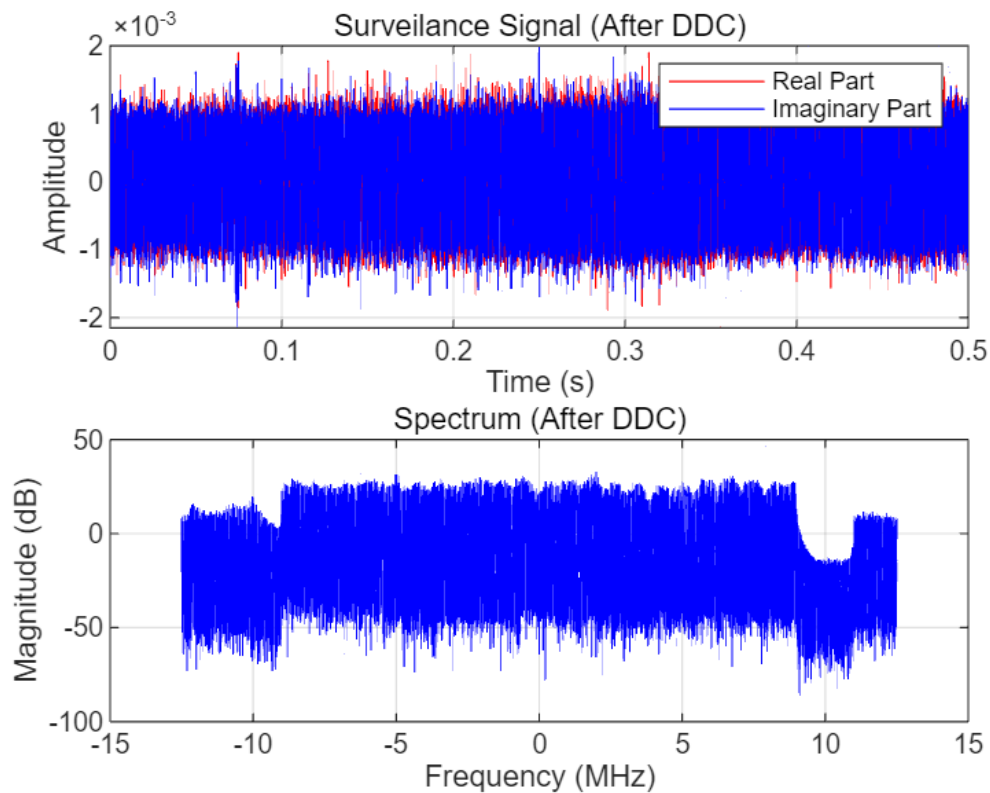


图 8 监测信号完整时域图+完整频域图(After DDC)

(3) 对两个信号继续做低通滤波处理，并画出时域频域图。

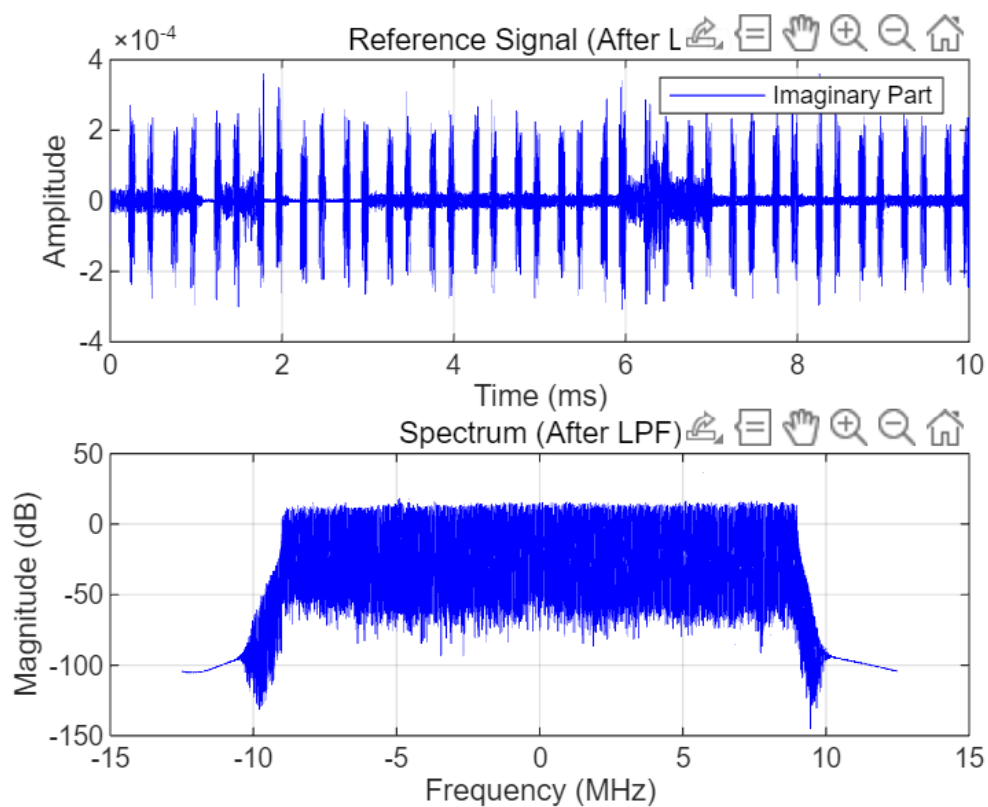


图9 参考信号时域图局部图（虚部）+完整频域图(After LPF)

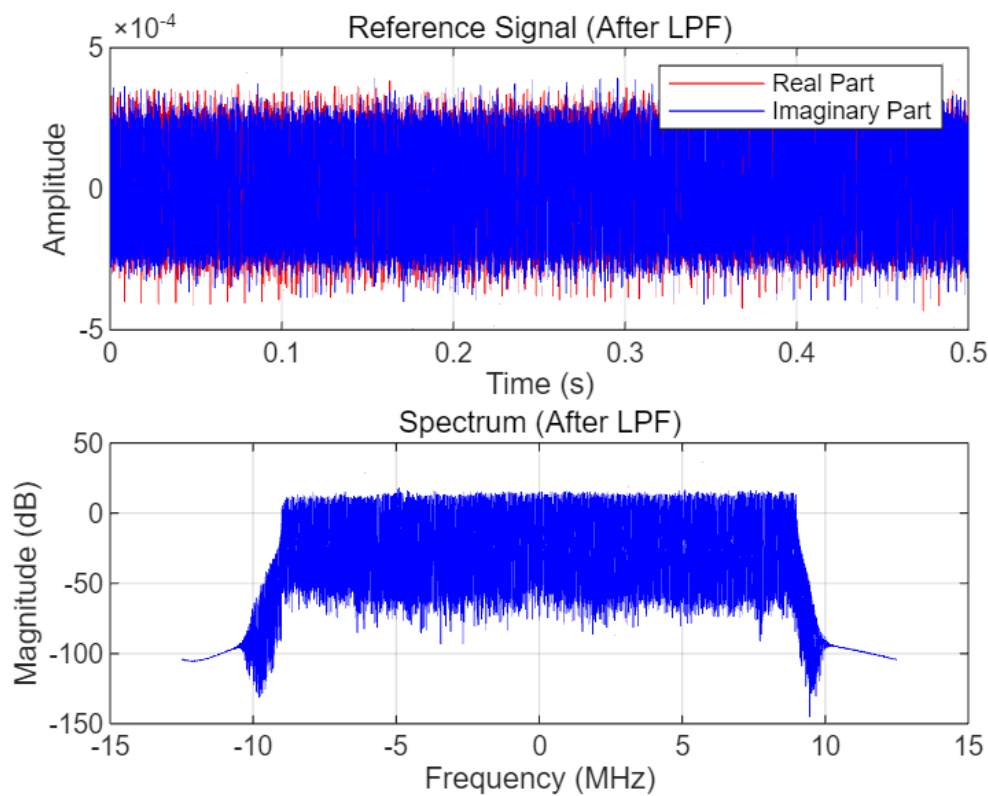


图10 参考信号完整时域图+完整频域图(After LPF)

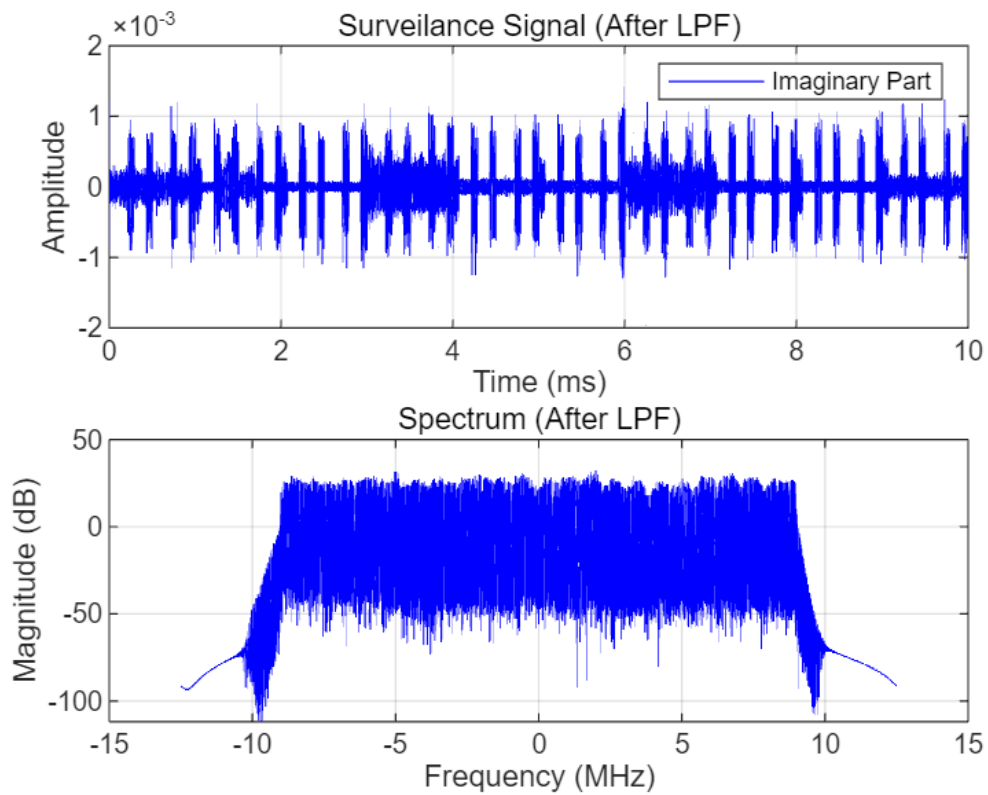


图11 监测信号时域图局部图（虚部）+完整频域图(After LPF)

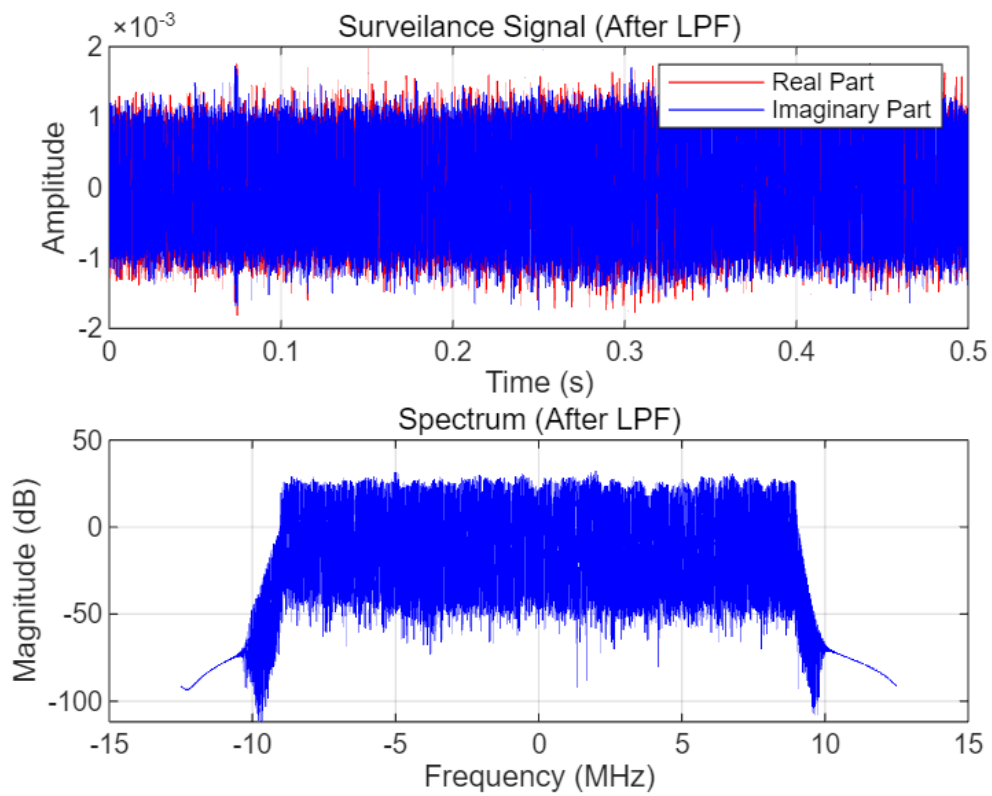


图12 监测信号完整时域图+完整频域图 (After LPF)

3. 关键代码

数据处理以参考信号为例：

```
function out = refer(n)

% 加载数据
addpath('data');
load(sprintf('data\\data_%d.mat', n));

% 参数设置
bandwidth = 9e6;
ts = 1/f_s;
duration_plot = 0.5;
num_t_axis_plot = duration_plot*f_s;
t_axis_plot = 0:1/f_s: duration_plot-1/f_s;
N = length(seq_ref);
f = (-N/2:N/2-1)*(f_s/N)/1e6;

% 初始信号频域
seq_ref_f = 20*log10(abs(fftshift(fft(seq_ref))));

% DDC
f_ddc = -3*1e6;
seq_ref_ddc_t = seq_ref .* exp(-1i*2*pi*f_ddc*(0:N-1)/f_s);
seq_ref_ddc_f = 20*log10(abs(fftshift(fft(seq_ref_ddc_t))));

% LPF
[b, a] = butter(20, bandwidth/(0.5*f_s));
seq_ref_lpf_t = filter(b, a, seq_ref_ddc_t);
seq_ref_lpf_f = 20*log10(abs(fftshift(fft(seq_ref_lpf_t))));

% 输出
out = seq_ref_lpf_f;
```

参考信号的时域局部（虚部）绘图：

```
% 绘图 - 初始信号
figure;
subplot(2,1,1);
%plot(t_axis_plot*1000, real(seq_ref), 'r');
%hold on;
plot(t_axis_plot*1000, imag(seq_ref), 'b');
grid on;
xlim([0,10]);
ylim([-4e-4,4e-4]);
xlabel('Time (ms)');
ylabel('Amplitude');
legend('Imaginary Part');
title('Reference Signal (Original)');
```

参考信号的时域完整绘图：

```
% 绘图 - 初始信号
figure;
subplot(2,1,1);
plot(t_axis_plot, real(seq_ref), 'r');
hold on;
plot(t_axis_plot, imag(seq_ref), 'b');
grid on;
xlabel('Time (s)');
ylabel('Amplitude');
legend('Real Part','Imaginary Part');
title('Reference Signal (Original)');
```

LPF的代码总览（测试）：

```
addpath('data');
load(sprintf('data\\data_%d.mat', 1));
bandwidth = 9e6;
[b, a] = butter(20, bandwidth/(0.5*f_s));
[H, f_H] = freqz(b, a, 4096, f_s);
figure;
plot(f_H*1e-6, 20*log10(abs(H)));
xlabel('Frequency (MHz)');
ylabel('Magnitude (dB)');
ylim([-3, 0]);
title('Frequency response of LPF');
```

(二) Task2:

1. 任务描述：

本任务的目标是计算出每个 0.5 秒时间窗口内的最合适的时间差 (τ) 和多普勒频移 (fD)。这些参数对于评估被检测目标与雷达之间的相对位置、距离及其运动速度至关重要。时间差 τ 反映了目标与雷达之间的距离，它基于信号传播的延时计算得出，而多普勒频移 fD 则能够揭示目标的相对运动速度，特别是在雷达与目标之间的相对运动导致的频率变化。

2. 实验过程：

(1) 数据读取与预处理

数据加载：从data文件夹中读取每个时间段的数据，每个数据包含了参考信号和监视信号。

数字下变频（DDC）：对信号进行数字下变频处理，减去频率偏移量，以便信号集中在低频带进行后续处理。

(2) 低通滤波处理

低通滤波（LPF）：使用butter函数设计一个带宽为bandwidth的低通滤波器，对下变频后的信号进行滤波，去除高频噪声，得到更加平滑的信号。

(3) 模糊函数处理

模糊函数用于处理信号的范围和多普勒频率。其目的是通过计算 相干函数 (correlation) 来找出最佳的时延 (τ) 和多普勒频率 (fD) 配对。

τ 的取值范围：根据采样频率 f_s 和信号传播速度 c 计算，得到最小时延为 $1/f_s$ ，最小

距离分辨率为 c/f_s 。由于目标物体运动不超过100m，故将时延（范围）设定为 [0, 72]m，即 array_sample_shift 为 [0:1:5]。

fD 的取值范围：根据最大速度限制计算多普勒频率的变化范围，设定为 array_Doppler_frequency = [-40:2:40] Hz。

模糊函数遍历：使用三重 for 循环遍历 时延 (τ) 和多普勒频率 (fD) 的每一个可能组合。每次计算信号在该参数组合下的 相干函数，然后选择出最大值对应的时延和多普勒频率。

通过这种方法，对于每一段数据，我们得到了一个 模糊函数矩阵，该矩阵包含了每种时延和多普勒频率组合的相关性。

```
for idx_RD = 1:num_loop
    idx_sample_shift = ceil(idx_RD/length(array_Doppler_frequency));
    idx_f_d = mod(idx_RD-1,length(array_Doppler_frequency))+1;
    sample_shift = array_sample_shift(idx_sample_shift);
    f_d = array_Doppler_frequency(idx_f_d);

    temp(1,idx_RD) = sum(seq_sur_lpf(1,1+sample_shift:end) ...
        .*conj(seq_ref_lpf(1,1:end-sample_shift)) ...
        .*exp(-1i*2*pi*f_d*t_axis(1+sample_shift:end)));

    A_TRD(idx_start_time, idx_sample_shift, idx_f_d) = abs(temp(1, idx_RD));
end
```

(4) 绘制 Range-Doppler Spectrum

在模糊函数处理后，使用 surf 函数绘制 Range-Doppler Spectrum。该图的横坐标是多普勒频率 (fD)，纵坐标是距离 (τ)，图中的亮度表示模糊函数值的大小。对应的最佳时延 (τ) 和多普勒频率 (fD) 即为图像中最亮的区域。

```
plot_A_RD = abs(squeeze(A_TRD(idx_start_time, :, :)));
plot_A_RD = plot_A_RD/max(max(plot_A_RD));
plot_A_RD = 20*log10(plot_A_RD);
plot_A_RD(plot_A_RD<thres_A_TRD) = thres_A_TRD;
plot_A_RD = [plot_A_RD;thres_A_TRD*ones(1,size(plot_A_RD,2))];
surf(meshgrid_Doppler,meshgrid_range,plot_A_RD)
```

(5) 绘制 Time-Doppler Spectrum

基于模糊函数矩阵的结果，绘制 Time-Doppler Spectrum，该图的横坐标为多普勒频率 (fD)，纵坐标为开始时间，亮度表示模糊函数值。

```
plot_A_TD = zeros(length(array_start_time),length(array_Doppler_frequency));
for idx_start_time = 1:length(array_start_time)
    plot_A_TD(idx_start_time,:) = abs(squeeze(A_TRD(idx_start_time,idx_max_range(idx_start_time),:)));
end
plot_A_TD = plot_A_TD./max(plot_A_TD,[],2);
plot_A_TD = 20*log10(plot_A_TD);
plot_A_TD(plot_A_TD<thres_A_TRD) = thres_A_TRD;
plot_A_TD = [plot_A_TD;thres_A_TRD*ones(1,size(plot_A_TD,2))];
surf(meshgrid_Doppler,meshgrid_start_time,plot_A_TD)
```

3. 作图:

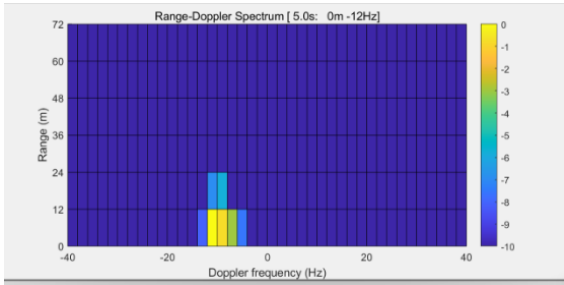


图13 0~0.5s

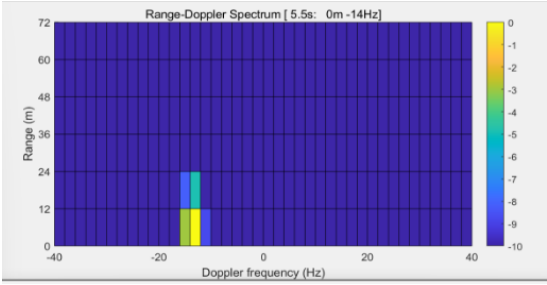


图14 2~2.5s

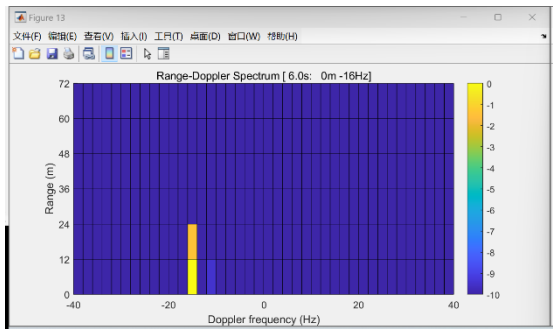


图15 5~5.5s

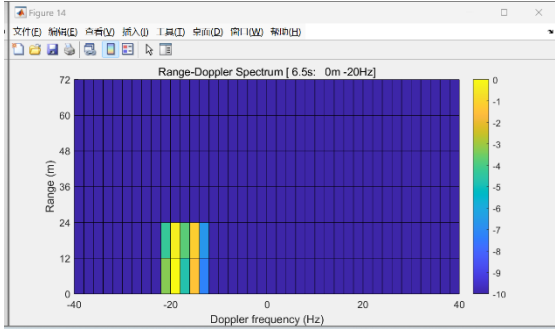


图16 7~7.5s

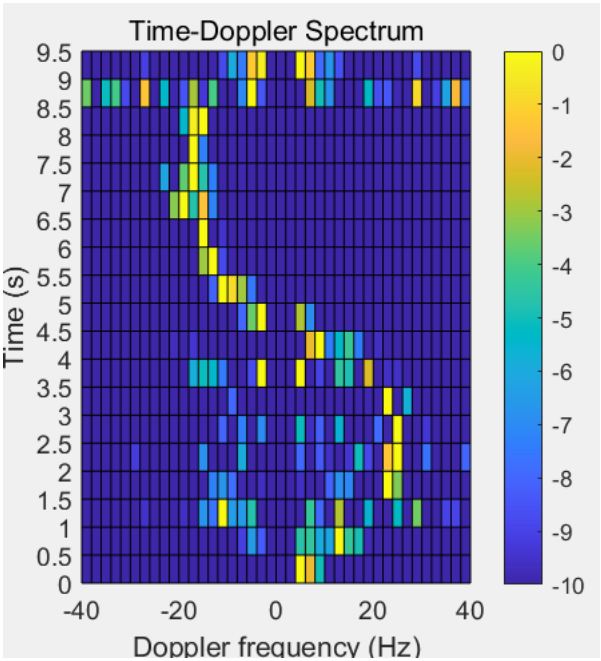


图17 最终的多普勒-时间图

(三) 拓展：优化模糊函数算法

1.优化方式：

(1)使用 fftshift 函数优化频谱对齐

在计算FFT结果时，使用了 `fftshift` 函数，它将FFT输出的频谱移到中心。通常，FFT的输出频率轴是从0到最大频率（正频率部分），然后是负频率部分。通过使用 `fftshift`，代码将频谱的零频率组件移到中心，这样可以使得频谱看起来对称，便于分析和后续处理。

优化效果：通过频谱的对称性，避免了对频率轴的手动处理，提高了分析频谱的可读性和效率。

(2)通过 conj 对信号进行复共轭处理

`seq_sur_lpf(1,1+sample_shift:end)` 和 `seq_ref_lpf(1,1:end-sample_shift)` 是两个信号序列，第二个信号 `seq_ref_lpf` 经过了复共轭处理。这通常用于计算互相关或者相关函数，尤其是在雷达信号处理中，这样可以最大化匹配的相关性。

优化效果：通过复共轭操作，代码能够实现信号之间的相关性计算，增强了信号的匹配和检测性能，特别是在多普勒频移的情况下。

(3)线性插值处理 Doppler 频率

使用 `interp1` 对 FFT 结果进行插值，特别是根据多普勒频率 `array_Doppler_frequency` 来进行插值，确保即使在 FFT 分辨率上没有精确匹配的频率点时，仍能得到合适的频率值。

优化效果：通过插值，确保了对多普勒频率的高精度采样，即使 `array_Doppler_frequency` 中的频率不在 FFT 结果的离散频率点上，插值也能够提供精确的值，避免了频率分辨率不足带来的误差。

(4)动态生成频率轴

使用 `linspace(-f_s/2, f_s/2, length(fft_result))` 动态生成频率轴 `f_axis_plot`，其中 `f_s` 是采样频率。这样频率轴的生成是根据 FFT 长度自动计算的，而不是固定的，确保了无论 FFT 的长度如何变化，频率轴总是正确的。

优化效果：自动计算频率轴，提高了代码的灵活性，避免了硬编码带来的错误。

(5)循环处理不同的样本偏移量

`for idx_sample_shift = 1:length(array_sample_shift)` 循环处理不同的样本偏移量，这种处理方式允许对不同的偏移量进行多次 FFT 计算，通常用于处理多普勒频移的影响，并评估不同的样本偏移对结果的影响。

优化效果：通过循环，能够对不同的样本偏移量进行全面分析，进一步提高了对信号特性的理解。

2.代码实现:

```
% FFT Ambiguity function
fprintf('[stat]   Ambiguity processing.\n')

for idx_sample_shift = 1:length(array_sample_shift)
    sample_shift = array_sample_shift(idx_sample_shift); %从数组[0:5]中取值

    fft_result = fftshift(fft(seq_sur_lpf(1,1+sample_shift:end).*conj(seq_ref_lpf(1,1:end-sample_shift))));
    f_axis_plot = linspace(-f_s/2, f_s/2, length(fft_result)); % 根据 FFT 长度动态生成频率轴
    A_TRD(idx_start_time, idx_sample_shift,:) = interp1(f_axis_plot, fft_result, array_Doppler_frequency, 'linear', 0);
end
```

3.运行结果:

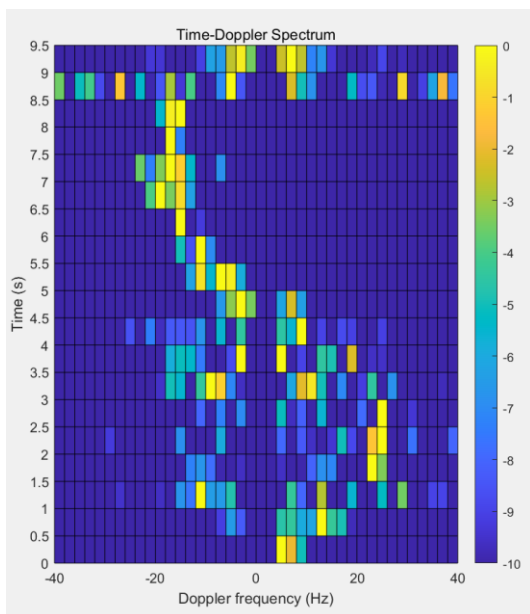


图18 fft

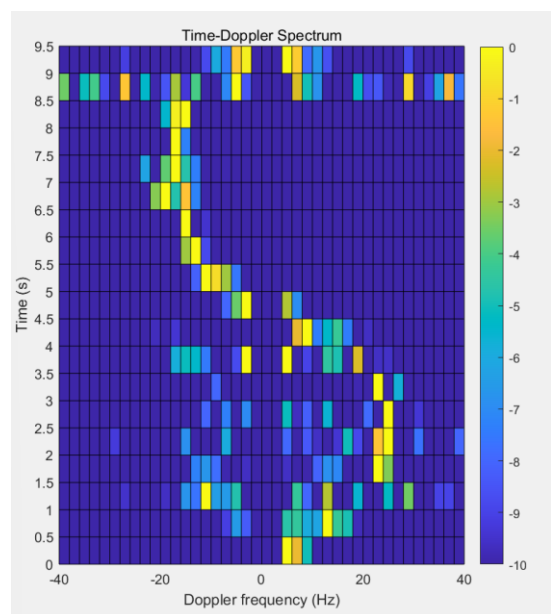


图19 dwt

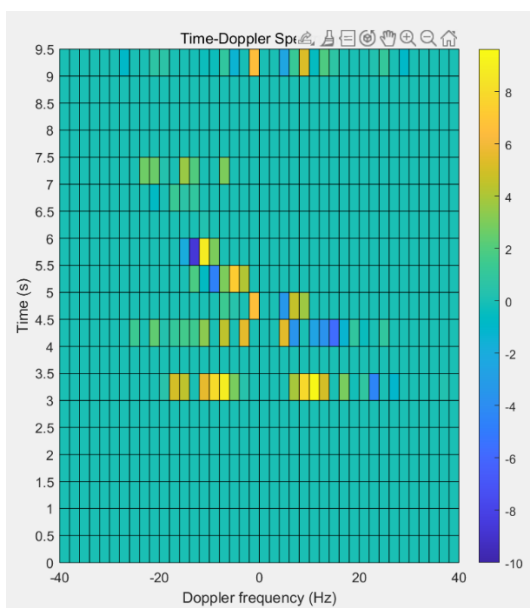


图20 DWT 循环算法与 FFT 算法结果对比

4.创新：以0.1s进行分割

(1) 代码实现：

```
%0.1s
for idx_start_time = 1:length(array_start_time)
    start_time = array_start_time(idx_start_time);
    file_idx = floor(start_time / 0.5) + 1; % 确定文件编号
    offset = mod(start_time, 0.5) * f_s; % 计算在文件内的偏移量

    % 加载当前及相邻文件数据
    fprintf('[stat] Index of start time: %d / %d.\n', idx_start_time, length(array_start_time))
    fprintf('[stat] Read data files.\n')
    load(sprintf('data/data_%d.mat', file_idx));

    % 提取所需的时间段数据
    seq_ref_window = seq_ref(1, 1 + offset:offset + duration_time * f_s);
    seq_sur_window = seq_sur(1, 1 + offset:offset + duration_time * f_s);
    % idx_start_time = 1;
    % idx_start_time

    % Digital downconvert %数字下变频
    fprintf('[stat] Downconvert.\n')
    seq_ref_ddc = seq_ref_window.*exp(-1i*2*pi*f_ddc*[0:duration_time*f_s-1]/f_s);
    seq_sur_ddc = seq_sur_window.*exp(-1i*2*pi*f_ddc*[0:duration_time*f_s-1]/f_s);
```

(2) 作图：

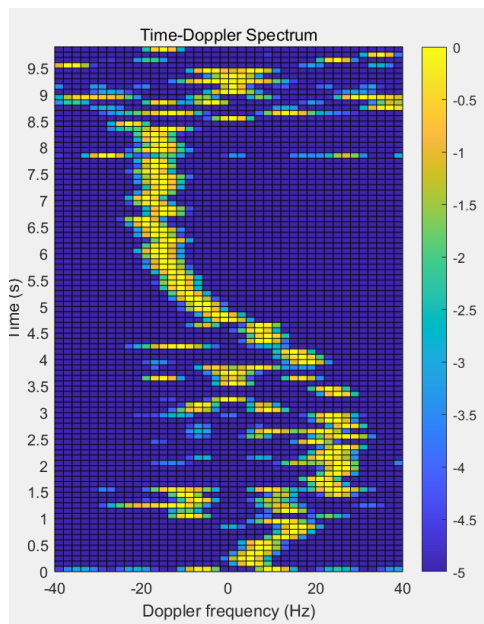


图21 fft

5.结论：

使用 FFT 算法能在保证结果正确的前提下大幅度降低运算时间，不失为一种高效的方法。

五. 实验收获

1. 参考信号与检测信号由于其本身复数信号的特性，其完整时域图图像复杂，难以清晰辨别其经过数字下变频与通过低通滤波器的信号变化。我们采用限制图像的x、y轴参数，以及将单位由s替换到ms，找出0-10ms的区间，观察信号的虚部时域图，从而能够清晰辨别信号经两个历程所发生的变化。
2. 本次project1周晓俊同学负责了程序的主要部分，模糊函数的定义，作图部分函数。在定义模糊函数过程中，我先尝试使用三轮for循环实现20轮6×41次的循环，但是跑一次完整的程序长达3小时，后续尝试参考老师的方法，使用两次取模和取整运算替代for循环，减少循环数量，时间缩短到25分钟左右，最后尝试使用fft的方法将求和预算转化为快速傅里叶变换，时间再次大幅缩短至三分钟就能跑完一次完整程序，后面进行结果比对，却发现fft得出信号与最开始dwt遍历有一些差距，又做误差图分析，在某些时间段内误差为零，在几个时间段内却又有着不同程度的误差，时间有限，未能得出具体原因。期间还尝试改变滤波器的不同阶数，比较不同阶数下的出的结果，未发现较大差异。还尝试将10秒的信号按0.1s进行分割（原来是0.5s），再进行100轮的求解，得出更细致的结果图（与老师0.5s滑动窗口不同）。
3. 通过小组分工合作的模式完成本次项目，增强了小组的团队凝聚力与个人的协调沟通能力，为未来的团队任务打下坚实的基础。

Score:

何衍宁 100

夏天 100

周晓俊 97

张安泰 100

Code:

```
function out = surveil(n)

% 加载数据

addpath('data');

load(sprintf('data\\data_%d.mat', n));
```



```
% 参数设置

bandwidth = 9e6;

ts = 1/f_s;

duration_plot = 0.5;

t_axis_plot = 0:1/f_s: duration_plot-1/f_s;

N = length(seq_sur);

f = (-N/2:N/2-1)*(f_s/N)/1e6;

% 初始信号频域

seq_sur_f = 20*log10(abs(fftshift(fft(seq_sur))));

% DDC

f_ddc = -3*1e6;

seq_sur_ddc_t = seq_sur .* exp(-1i*2*pi*f_ddc*(0:N-1)/f_s);

seq_sur_ddc_f = 20*log10(abs(fftshift(fft(seq_sur_ddc_t))));

% LPF

[b, a] = butter(20, bandwidth/(0.5*f_s));

seq_sur_lpf_t = filter(b, a, seq_sur_ddc_t);
```

```
seq_sur_lpf_f = 20*log10(abs(fftshift(fft(seq_sur_lpf_t))));
```

```
% 输出
```

```
out = seq_sur_lpf_f;
```

```
% 绘图 - 初始信号
```

```
figure;
```

```
subplot(2,1,1);
```

```
%plot(t_axis_plot, real(seq_sur), 'r');
```

```
%hold on;
```

```
plot(t_axis_plot*1000, imag(seq_sur), 'b');
```

```
grid on;
```

```
xlim([0,10]);
```

```
ylim([-2e-3,2e-3]);
```

```
xlabel('Time (ms)');
```

```
ylabel('Amplitude');
```

```
legend('Imaginary Part');
```

```
title('Surveillance Signal (Original)');

subplot(2,1,2);

plot(f, seq_sur_f, 'b');

grid on;

xlabel('Frequency (MHz)');

ylabel('Magnitude (dB)');

title('Spectrum (Original)');

% 绘图 - DDC

figure;

subplot(2,1,1);

%plot(t_axis_plot, real(seq_sur_ddc_t), 'r');

%hold on;

plot(t_axis_plot*1000, imag(seq_sur_ddc_t), 'b');

grid on;
```

```
xlim([0,10]);

ylim([-2e-3,2e-3]);

xlabel('Time (ms)');

ylabel('Amplitude');

legend('Imaginary Part');

title('Surveillance Signal (After DDC)');


subplot(2,1,2);

plot(f, seq_sur_ddc_f, 'b');

grid on;

xlabel('Frequency (MHz)');

ylabel('Magnitude (dB)');

title('Spectrum (After DDC)');


% 绘图 - LPF

figure;

subplot(2,1,1);
```

```
%plot(t_axis_plot, real(seq_sur_lpf_t), 'r');

%hold on;

plot(t_axis_plot*1000, imag(seq_sur_lpf_t), 'b');

grid on;

xlim([0,10]);

ylim([-2e-3,2e-3]);

xlabel('Time (ms)');

ylabel('Amplitude');

legend( 'Imaginary Part');

title('Surveillance Signal (After LPF)');

subplot(2,1,2);

plot(f, seq_sur_lpf_f, 'b');

grid on;

xlabel('Frequency (MHz)');

ylabel('Magnitude (dB)');
```

```
title('Spectrum (After LPF)');
```

```
end
```

```
function out = surveilri(n)
```

```
% 加载数据
```

```
addpath('data');
```

```
load(sprintf('data\\data_%d.mat', n));
```

```
% 参数设置
```

```
bandwidth = 9e6;
```

```
ts = 1/f_s;
```

```
duration_plot = 0.5;
```

```
t_axis_plot = 0:1/f_s: duration_plot-1/f_s;
```

```
N = length(seq_sur);
```

```
f = (-N/2:N/2-1)*(f_s/N)/1e6;
```

```
% 初始信号频域
```

```
seq_sur_f = 20*log10(abs(fftshift(fft(seq_sur))));

% DDC

f_ddc = -3*1e6;

seq_sur_ddc_t = seq_sur .* exp(-1i*2*pi*f_ddc*(0:N-1)/f_s);

seq_sur_ddc_f = 20*log10(abs(fftshift(fft(seq_sur_ddc_t))));

% LPF

[b, a] = butter(20, bandwidth/(0.5*f_s));

seq_sur_lpf_t = filter(b, a, seq_sur_ddc_t);

seq_sur_lpf_f = 20*log10(abs(fftshift(fft(seq_sur_lpf_t))));

% 输出

out = seq_sur_lpf_f;

% 绘图 - 初始信号

figure;

subplot(2,1,1);

plot(t_axis_plot, real(seq_sur), 'r');

hold on;
```

```
plot(t_axis_plot, imag(seq_sur), 'b');
```

```
grid on;
```

```
xlabel('Time (s)');
```

```
ylabel('Amplitude');
```

```
legend('Real Part','Imaginary Part');
```

```
title('Surveillance Signal (Original)');
```

```
subplot(2,1,2);
```

```
plot(f, seq_sur_f, 'b');
```

```
grid on;
```

```
xlabel('Frequency (MHz)');
```

```
ylabel('Magnitude (dB)');
```

```
title('Spectrum (Original)');
```

```
% 绘图 - DDC
```

```
figure;
```



```
subplot(2,1,1);

plot(t_axis_plot, real(seq_sur_ddc_t), 'r');

hold on;

plot(t_axis_plot, imag(seq_sur_ddc_t), 'b');

grid on;

xlabel('Time (s)');

ylabel('Amplitude');

legend('Real Part','Imaginary Part');

title('Surveillance Signal (After DDC)');

subplot(2,1,2);

plot(f, seq_sur_ddc_f, 'b');

grid on;

xlabel('Frequency (MHz)');

ylabel('Magnitude (dB)');

title('Spectrum (After DDC)');
```

```
% 绘图 - LPF
```

```
figure;
```

```
subplot(2,1,1);
```

```
plot(t_axis_plot, real(seq_sur_lpf_t), 'r');
```

```
hold on;
```

```
plot(t_axis_plot, imag(seq_sur_lpf_t), 'b');
```

```
grid on;
```

```
xlabel('Time (s)');
```

```
ylabel('Amplitude');
```

```
legend('Real Part', 'Imaginary Part');
```

```
title('Surveillance Signal (After LPF)');
```

```
subplot(2,1,2);
```

```
plot(f, seq_sur_lpf_f, 'b');
```

```
grid on;
```

```
xlabel('Frequency (MHz)');

ylabel('Magnitude (dB)');

title('Spectrum (After LPF)');

end

function out = refer(n)

% 加载数据

addpath('data');

load(sprintf('data\\data_%d.mat', n));

% 参数设置

bandwidth = 9e6;

ts = 1/f_s;

duration_plot = 0.5;

num_t_axis_plot = duration_plot*f_s;

t_axis_plot = 0:1/f_s: duration_plot-1/f_s;

N = length(seq_ref);

f = (-N/2:N/2-1)*(f_s/N)/1e6;
```

```
% 初始信号频域

seq_ref_f = 20*log10(abs(fftshift(fft(seq_ref))));

% DDC

f_ddc = -3*1e6;

seq_ref_ddc_t = seq_ref .* exp(-1i*2*pi*f_ddc*(0:N-1)/f_s);

seq_ref_ddc_f = 20*log10(abs(fftshift(fft(seq_ref_ddc_t))));

% LPF

[b, a] = butter(20, bandwidth/(0.5*f_s));

seq_ref_lpf_t = filter(b, a, seq_ref_ddc_t);

seq_ref_lpf_f = 20*log10(abs(fftshift(fft(seq_ref_lpf_t))));

% 输出

out = seq_ref_lpf_f;

% 绘图 - 初始信号

figure;

subplot(2,1,1);
```

```
%plot(t_axis_plot*1000, real(seq_ref), 'r');
```

```
%hold on;
```

```
plot(t_axis_plot*1000, imag(seq_ref), 'b');
```

```
grid on;
```

```
xlim([0,10]);
```

```
ylim([-4e-4,4e-4]);
```

```
xlabel('Time (ms)');
```

```
ylabel('Amplitude');
```

```
legend('Imaginary Part');
```

```
title('Reference Signal (Original)');
```

```
subplot(2,1,2);
```

```
plot(f, seq_ref_f, 'b');
```

```
grid on;
```

```
xlabel('Frequency (MHz)');
```

```
ylabel('Magnitude (dB)');
```

```
title('Spectrum (Original)');

% 绘图 - DDC

figure;

subplot(2,1,1);

%plot(t_axis_plot*1000, real(seq_ref_ddc_t), 'r');

%hold on;

plot(t_axis_plot*1000, imag(seq_ref_ddc_t), 'b');

grid on;

xlim([0,10]);

ylim([-4e-4,4e-4]);

xlabel('Time (ms)');

ylabel('Amplitude');

legend('Imaginary Part');

title('Reference Signal (After DDC)');
```

```
subplot(2,1,2);

plot(f, seq_ref_ddc_f, 'b');

grid on;

xlabel('Frequency (MHz)');

ylabel('Magnitude (dB)');

title('Spectrum (After DDC)');


% 绘图 - LPF

figure;

subplot(2,1,1);

%plot(t_axis_plot, real(seq_ref_lpf_t), 'r');

%hold on;

plot(t_axis_plot*1000, imag(seq_ref_lpf_t), 'b');

grid on;

xlim([0,10]);

ylim([-4e-4,4e-4]);
```

```
xlabel('Time (ms)');

ylabel('Amplitude');

legend('Imaginary Part');

title('Reference Signal (After LPF)');

subplot(2,1,2);

plot(f, seq_ref_lpf_f, 'b');

grid on;

xlabel('Frequency (MHz)');

ylabel('Magnitude (dB)');

title('Spectrum (After LPF)');

end

function out = referri(n)

% 加载数据

addpath('data');

load(sprintf('data\\data_%d.mat', n));

% 参数设置
```



```
bandwidth = 9e6;

ts = 1/f_s;

duration_plot = 0.5;

num_t_axis_plot = duration_plot*f_s;

t_axis_plot = 0:1/f_s: duration_plot-1/f_s;

N = length(seq_ref);

f = (-N/2:N/2-1)*(f_s/N)/1e6;

% 初始信号频域

seq_ref_f = 20*log10(abs(fftshift(fft(seq_ref))));

% DDC

f_ddc = -3*1e6;

seq_ref_ddc_t = seq_ref .* exp(-1i*2*pi*f_ddc*(0:N-1)/f_s);

seq_ref_ddc_f = 20*log10(abs(fftshift(fft(seq_ref_ddc_t))));

% LPF

[b, a] = butter(20, bandwidth/(0.5*f_s));
```

```
seq_ref_lpf_t = filter(b, a, seq_ref_ddc_t);

seq_ref_lpf_f = 20*log10(abs(fftshift(fft(seq_ref_lpf_t))));

% 输出

out = seq_ref_lpf_f;

% 绘图 - 初始信号

figure;

subplot(2,1,1);

plot(t_axis_plot, real(seq_ref), 'r');

hold on;

plot(t_axis_plot, imag(seq_ref), 'b');

grid on;

xlabel('Time (s)');

ylabel('Amplitude');

legend('Real Part','Imaginary Part');

title('Reference Signal (Original)');
```

```
subplot(2,1,2);
```

```
plot(f, seq_ref_f, 'b');
```

```
grid on;
```

```
xlabel('Frequency (MHz)');
```

```
ylabel('Magnitude (dB)');
```

```
title('Spectrum (Original)');
```

```
% 绘图 - DDC
```

```
figure;
```

```
subplot(2,1,1);
```

```
plot(t_axis_plot, real(seq_ref_ddc_t), 'r');
```

```
hold on;
```

```
plot(t_axis_plot, imag(seq_ref_ddc_t), 'b');
```

```
grid on;
```

```
xlabel('Time (s)');
```

```
ylabel('Amplitude');

legend('Real Part','Imaginary Part');

title('Reference Signal (After DDC)');


subplot(2,1,2);

plot(f, seq_ref_ddc_f, 'b');

grid on;

xlabel('Frequency (MHz)');

ylabel('Magnitude (dB)');

title('Spectrum (After DDC)');


% 绘图 - LPF

figure;

subplot(2,1,1);

plot(t_axis_plot, real(seq_ref_lpf_t), 'r');

hold on;

plot(t_axis_plot, imag(seq_ref_lpf_t), 'b');
```

```
grid on;

xlabel('Time (s)');

ylabel('Amplitude');

legend('Real Part','Imaginary Part');

title('Reference Signal (After LPF)');


subplot(2,1,2);

plot(f, seq_ref_lpf_f, 'b');

grid on;

xlabel('Frequency (MHz)');

ylabel('Magnitude (dB)');

title('Spectrum (After LPF)');

end

addpath('data');

load(sprintf('data\\data_%d.mat', 1));

bandwidth = 9e6;
```

```
[b, a] = butter(20, bandwidth/(0.5*f_s));
```

```
[H,f_H] = freqz(b,a,4096,f_s);
```

```
figure;
```

```
plot(f_H*1e-6,20*log10(abs(H)));
```

```
xlabel('Frequency (MHz)');
```

```
ylabel('Magnitude (dB)');
```

```
ylim([-3,0]);
```

```
title('Frequenct response of LPF');
```

```
clear;
```

```
close all;
```

```
clc;
```

```
addpath('data')
```

```
ScreenSize = get(0,'ScreenSize');
```

```
array_start_time = [0:0.5:9.5];
```

%每隔 0.5s 截取一段数据

```
% array_start_time = [0:0.1:9.9];
```

%每隔 0.5s 截取一段数据

```
array_sample_shift = [0:1:5];
```

%时移范围[0 5]

```
array_Doppler_frequency = [-40:2:40];
```

%频移范围[-40 40]

```
f_c = 2.123e9;
```

%载波频率 2.123GHz

```
f_s = 25e6;
```

%采样率 25MHz

```

lambda = 3e8/f_c; %波长

array_range = (array_sample_shift/f_s)*3e8; %时差*光速=距离范围

f_ddc = -3e6;

bandwidth = 9e6;

duration_time = 0.5/1;

A_TRD = zeros(length(array_start_time),length(array_sample_shift),length(array_Doppler_frequency));

%%0.1s

% for idx_start_time = 1:length(array_start_time)

%     start_time = array_start_time(idx_start_time);

%     file_idx = floor(start_time / 0.5) + 1; % 确定文件编号

%     offset = mod(start_time, 0.5) * f_s; % 计算在文件内的偏移量

%

%     % 加载当前及相邻文件数据

%     fprintf('[stat] Index of start time: %d / %d.\n', idx_start_time, length(array_start_time))

%     fprintf('[stat]   Read data files.\n')

%     load(sprintf('data/data_%d.mat', file_idx));

%

%     % 提取所需的时间段数据

%     seq_ref_window = seq_ref(1, 1 + offset:offset + duration_time * f_s);

%     seq_sur_window = seq_sur(1, 1 + offset:offset + duration_time * f_s);

%     % idx_start_time = 1;

%     % idx_start_time

%

%     %% Digital downconvert %数字下变频

%     fprintf('[stat]   Downconvert.\n')

%     seq_ref_ddc = seq_ref_window.*exp(-1i*2*pi*f_ddc*[0:duration_time*f_s-1]/f_s);

%     seq_sur_ddc = seq_sur_window.*exp(-1i*2*pi*f_ddc*[0:duration_time*f_s-1]/f_s);

%%0.5s

```

```

for idx_start_time = 1:length(array_start_time)

    % idx_start_time = 1;

    % idx_start_time

    fprintf('[stat] Index of start time: %d / %d.\n',idx_start_time,length(array_start_time))

    %% Read Data File                                %读取第一段 5s 数据

    fprintf('[stat]   Read data file.\n')

    load(sprintf('data/data_%d.mat',idx_start_time))

    %% Digital downconvert                            %数字下变频

    fprintf('[stat]   Downconvert.\n')

    seq_ref_ddc = seq_ref.*exp(-1i*2*pi*f_ddc*[0:duration_time*f_s-1]/f_s);

    seq_sur_ddc = seq_sur.*exp(-1i*2*pi*f_ddc*[0:duration_time*f_s-1]/f_s);

    %% LPF                                            %低通滤波器

    fprintf('[stat]   LPF.\n')

    [b,a] = butter(20,bandwidth/(f_s/2));

    seq_ref_lpf = filter(b,a,seq_ref_ddc);

    seq_sur_lpf = filter(b,a,seq_sur_ddc);

    %% Plot waveform and spectrum                    %画出时域波形和频谱

    duration_plot = 0.01;

    num_t_axis_plot = duration_plot*f_s;

    num_f_axis_plot = num_t_axis_plot;

    t_axis_plot = 0:1/f_s:duration_plot-1/f_s;

    f_axis_plot = -f_s/2:f_s/(num_f_axis_plot-1):f_s/2;

    fig1 = figure(1);

    set(fig1,'Position',[1,41,0.4*ScreenSize(3),0.8*ScreenSize(4)]);

    subplot(6,2,1)

        plot(t_axis_plot*1e3,real(seq_ref(1,1:num_t_axis_plot)))

```



```

xlabel('Time (ms)')

ylabel('Amplitude')

axis([0,duration_plot*1e3,-0.5e-3,0.5e-3])

title('Reference signal (raw)')

subplot(6,2,2)

plot(f_axis_plot/1e6,20*log10(abs(fftshift(fft(seq_ref(1,1:num_t_axis_plot))))))

xlabel('Frequency (MHz)')

ylabel('Amplitude (dB)')

axis([-f_s/2/1e6,f_s/2/1e6,-100,0])

title('Spectrum (raw)')

subplot(6,2,3)

plot(t_axis_plot*1e3,real(seq_ref_ddc(1,1:num_t_axis_plot)))

xlabel('Time (ms)')

ylabel('Amplitude')

axis([0,duration_plot*1e3,-0.5e-3,0.5e-3])

title('Reference signal (after DDC)')

subplot(6,2,4)

plot(f_axis_plot/1e6,20*log10(abs(fftshift(fft(seq_ref_ddc(1,1:num_t_axis_plot))))))

xlabel('Frequency (MHz)')

ylabel('Amplitude (dB)')

axis([-f_s/2/1e6,f_s/2/1e6,-100,0])

title('Spectrum (after DDC)')

subplot(6,2,5)

plot(t_axis_plot*1e3,real(seq_ref_lpf(1,1:num_t_axis_plot)))

xlabel('Time (ms)')

ylabel('Amplitude')

axis([0,duration_plot*1e3,-0.5e-3,0.5e-3])

title('Reference signal (after LPF)')

subplot(6,2,6)

plot(f_axis_plot/1e6,20*log10(abs(fftshift(fft(seq_ref_lpf(1,1:num_t_axis_plot))))))

xlabel('Frequency (MHz)')

```

```

        ylabel('Amplitude (dB)')

        axis([-f_s/2/1e6,f_s/2/1e6,-100,0])

        title('Spectrum (after LPF)')

subplot(6,2,7)

        plot(t_axis_plot*1e3,real(seq_sur(1,1:num_t_axis_plot)))

        xlabel('Time (ms)')

        ylabel('Amplitude')

        axis([0,duration_plot*1e3,-2e-3,2e-3])

        title('Surveillance signal (raw)')

subplot(6,2,8)

        plot(f_axis_plot/1e6,20*log10(abs(fftshift(fft(seq_sur(1,1:num_t_axis_plot))))))

        xlabel('Frequency (MHz)')

        ylabel('Amplitude (dB)')

        axis([-f_s/2/1e6,f_s/2/1e6,-100,0])

        title('Spectrum (raw)')

subplot(6,2,9)

        plot(t_axis_plot*1e3,real(seq_sur_ddc(1,1:num_t_axis_plot)))

        xlabel('Time (ms)')

        ylabel('Amplitude')

        axis([0,duration_plot*1e3,-2e-3,2e-3])

        title('Surveillance signal (after DDC)')

subplot(6,2,10)

        plot(f_axis_plot/1e6,20*log10(abs(fftshift(fft(seq_sur_ddc(1,1:num_t_axis_plot))))))

        xlabel('Frequency (MHz)')

        ylabel('Amplitude (dB)')

        axis([-f_s/2/1e6,f_s/2/1e6,-100,0])

        title('Spectrum (after DDC)')

subplot(6,2,11)

        plot(t_axis_plot*1e3,real(seq_sur_lpf(1,1:num_t_axis_plot)))

        xlabel('Time (ms)')

        ylabel('Amplitude')

```

```

axis([0,duration_plot*1e3,-2e-3,2e-3])

title('Surveillance signal (after LPF)')

subplot(6,2,12)

plot(f_axis_plot/1e6,20*log10(abs(fftshift(fft(seq_sur_lpf(1,1:num_t_axis_plot))))))

xlabel('Frequency (MHz)')

ylabel('Amplitude (dB)')

axis([-f_s/2/1e6,f_s/2/1e6,-100,0])

title('Spectrum (after LPF)')

%% DWT Ambiguity function

fprintf('[stat]   Ambiguity processing.\n')

num_loop = length(array_sample_shift)*length(array_Doppler_frequency);

t_axis = 0:1/f_s:(duration_time*f_s-1)/f_s;

temp = zeros(1,num_loop);

for idx_RD = 1:num_loop

    idx_sample_shift = ceil(idx_RD/length(array_Doppler_frequency));

    idx_f_d = mod(idx_RD-1,length(array_Doppler_frequency))+1;

    sample_shift = array_sample_shift(idx_sample_shift);           %从数组[0:5]中取值

    f_d = array_Doppler_frequency(idx_f_d);                         %从数组[-40:2:40]中取值

    temp(1,idx_RD) = sum(seq_sur_lpf(1,1+sample_shift:end) ...

        .*conj(seq_ref_lpf(1,1:end-sample_shift)) ...

        .*exp(-1i*2*pi*f_d*t_axis(1+sample_shift:end)));

    A_TRD(idx_start_time, idx_sample_shift, idx_f_d) = abs(temp(1, idx_RD));

end

%% FFT Ambiguity function

% fprintf('[stat]   Ambiguity processing.\n')

%
```

```

% for idx_sample_shift = 1:length(array_sample_shift)

%     sample_shift = array_sample_shift(idx_sample_shift);           %从数组[0:5]中取值

%

%     fft_result = fftshift(fft(seq_sur_lpf(1,1+sample_shift:end).*conj(seq_ref_lpf(1,1:end-
sample_shift))));

%     f_axis_plot = linspace(-f_s/2, f_s/2, length(fft_result)); % 根据 FFT 长度动态生成频率轴

%     A_TRD(idx_start_time, idx_sample_shift,:) = interp1(f_axis_plot, fft_result,
array_Doppler_frequency, 'linear', 0);

% end

end

% Plot RD Spectrum

idx_max_range = zeros(1,length(array_start_time))-1;

idx_max_Doppler_frequency = zeros(1,length(array_start_time));

thres_A_TRD = -10;

for idx_start_time = 1:length(array_start_time+1)

    fig = figure(idx_start_time);

    ScreenSize = get(0,'ScreenSize');

    set(fig,'Position',[0.75*ScreenSize(3)+50,0.5*ScreenSize(4)+50,0.25*ScreenSize(3)-
100,0.5*ScreenSize(4)-150]);

    [meshgrid_Doppler,meshgrid_range] =
meshgrid(array_Doppler_frequency,[array_range,2*array_range(end)-array_range(end-1)]);

    plot_A_RD = abs(squeeze(A_TRD(idx_start_time,:,:)));

    plot_A_RD = plot_A_RD/max(max(plot_A_RD));

    plot_A_RD = 20*log10(plot_A_RD);

    plot_A_RD(plot_A_RD<thres_A_TRD) = thres_A_TRD;

    plot_A_RD = [plot_A_RD;thres_A_TRD*ones(1,size(plot_A_RD,2))];

    surf(meshgrid_Doppler,meshgrid_range,plot_A_RD)

    view(0,90)

    colorbar

    xlim([array_Doppler_frequency(1),array_Doppler_frequency(end)])

    ylim([array_range(1),2*array_range(end)-array_range(end-1)])

    xticks([array_Doppler_frequency(1):20:array_Doppler_frequency(end)])

    yticks([array_range,2*array_range(end)-array_range(end-1)])

```

```

        xlabel('Doppler frequency (Hz)')

        ylabel('Range (m)')

        [idx_max_range(idx_start_time),idx_max_Doppler_frequency(idx_start_time)] =
        find(plot_A_RD==max(max(plot_A_RD)));

        temp = sprintf('Range-Doppler Spectrum [%4.1fs: %3.0fm %3.0fHz]', ...

            array_start_time(idx_start_time), ...

            array_range(idx_max_range(idx_start_time)), ...

            array_Doppler_frequency(idx_max_Doppler_frequency(idx_start_time)));

        title(temp)

end

% Plot TD Spectrum

fig3 = figure(102);

ScreenSize = get(0,'ScreenSize');

set(fig3,'Position',[0.5*ScreenSize(3)+50,50,0.25*ScreenSize(3)-100,0.5*ScreenSize(4)-150]);

[meshgrid_Doppler,meshgrid_start_time] = ...

    meshgrid(array_Doppler_frequency,[array_start_time,array_start_time(end)+duration_time]);

plot_A_TD = zeros(length(array_start_time),length(array_Doppler_frequency));

for idx_start_time = 1:length(array_start_time)

    plot_A_TD(idx_start_time,:) =
    abs(squeeze(A_TRD(idx_start_time,idx_max_range(idx_start_time),:)));

end

plot_A_TD = plot_A_TD./max(plot_A_TD,[],2);

plot_A_TD = 20*log10(plot_A_TD);

plot_A_TD(plot_A_TD<thres_A_TRD) = thres_A_TRD;

plot_A_TD = [plot_A_TD;thres_A_TRD*ones(1,size(plot_A_TD,2))];

surf(meshgrid_Doppler,meshgrid_start_time,plot_A_TD)

view(0,90)

colorbar

xlim([array_Doppler_frequency(1),array_Doppler_frequency(end)])

ylim([array_start_time(1),array_start_time(end)])

xticks([array_Doppler_frequency(1):20:array_Doppler_frequency(end)])

```

```

yticks([array_start_time(1):0.5:array_start_time(end)+duration_time])

xlabel('Doppler frequency (Hz)')

ylabel('Time (s)')

title('Time-Doppler Spectrum')


close all;

clc;

addpath('data')

ScreenSize = get(0,'ScreenSize');


array_start_time = [0:0.5:9.5]; %每隔 0.5s 截取一段数据
array_sample_shift = [0:1:5]; %时移范围[0 5]
array_Doppler_frequency = [-40:2:40]; %频移范围[-40 40]


f_c = 2.123e9; %载波频率 2.123GHz
f_s = 25e6; %采样率 25MHz
lambda = 3e8/f_c; %波长
array_range = (array_sample_shift/f_s)*3e8; %时差*光速=距离范围
f_ddc = -3e6;
bandwidth = 9e6;


A_TRD_dwt =
zeros(length(array_start_time),length(array_sample_shift),length(array_Doppler_frequency));

A_TRD_fft =
zeros(length(array_start_time),length(array_sample_shift),length(array_Doppler_frequency));


for idx_start_time = 1:length(array_start_time)

    % idx_start_time = 1;

    % idx_start_time

    fprintf('[stat] Index of start time: %d / %d.\n',idx_start_time,length(array_start_time))

    %% Read Data File %读取第一段 5s 数据

```

```

fprintf('[stat]   Read data file.\n')

load(sprintf('data/data_%d.mat',idx_start_time))

%% Digital downconvert                                %数字下变频

fprintf('[stat]   Downconvert.\n')

seq_ref_ddc = seq_ref.*exp(-1i*2*pi*f_ddc*[0:duration*f_s-1]/f_s);

seq_sur_ddc = seq_sur.*exp(-1i*2*pi*f_ddc*[0:duration*f_s-1]/f_s);

%% LPF                                                %低通滤波器

fprintf('[stat]   LPF.\n')

[b,a] = butter(20,bandwidth/(f_s/2));

seq_ref_lpf = filter(b,a,seq_ref_ddc);

seq_sur_lpf = filter(b,a,seq_sur_ddc);

%% Plot waveform and spectrum                        %画出时域波形和频谱

duration_plot = 0.01;

num_t_axis_plot = duration_plot*f_s;

num_f_axis_plot = num_t_axis_plot;

t_axis_plot = 0:1/f_s:duration_plot-1/f_s;

f_axis_plot = -f_s/2:f_s/(num_f_axis_plot-1):f_s/2;

fig1 = figure(1);

set(fig1,'Position',[1,41,0.4*ScreenSize(3),0.8*ScreenSize(4)]);

subplot(6,2,1)

    plot(t_axis_plot*1e3,real(seq_ref(1,1:num_t_axis_plot)))

    xlabel('Time (ms)')

    ylabel('Amplitude')

    axis([0,duration_plot*1e3,-0.5e-3,0.5e-3])

    title('Reference signal (raw)')

subplot(6,2,2)

```

```

plot(f_axis_plot/1e6,20*log10(abs(fftshift(fft(seq_ref(1,1:num_t_axis_plot))))))

xlabel('Frequency (MHz)')

ylabel('Amplitude (dB)')

axis([-f_s/2/1e6,f_s/2/1e6,-100,0])

title('Spectrum (raw)')

subplot(6,2,3)

plot(t_axis_plot*1e3,real(seq_ref_ddc(1,1:num_t_axis_plot)))

xlabel('Time (ms)')

ylabel('Amplitude')

axis([0,duration_plot*1e3,-0.5e-3,0.5e-3])

title('Reference signal (after DDC)')

subplot(6,2,4)

plot(f_axis_plot/1e6,20*log10(abs(fftshift(fft(seq_ref_ddc(1,1:num_t_axis_plot))))))

xlabel('Frequency (MHz)')

ylabel('Amplitude (dB)')

axis([-f_s/2/1e6,f_s/2/1e6,-100,0])

title('Spectrum (after DDC)')

subplot(6,2,5)

plot(t_axis_plot*1e3,real(seq_ref_lpf(1,1:num_t_axis_plot)))

xlabel('Time (ms)')

ylabel('Amplitude')

axis([0,duration_plot*1e3,-0.5e-3,0.5e-3])

title('Reference signal (after LPF)')

subplot(6,2,6)

plot(f_axis_plot/1e6,20*log10(abs(fftshift(fft(seq_ref_lpf(1,1:num_t_axis_plot))))))

xlabel('Frequency (MHz)')

ylabel('Amplitude (dB)')

axis([-f_s/2/1e6,f_s/2/1e6,-100,0])

title('Spectrum (after LPF)')

subplot(6,2,7)

plot(t_axis_plot*1e3,real(seq_sur(1,1:num_t_axis_plot)))

```



```

xlabel('Time (ms)')

ylabel('Amplitude')

axis([0,duration_plot*1e3,-2e-3,2e-3])

title('Surveillance signal (raw)')

subplot(6,2,8)

plot(f_axis_plot/1e6,20*log10(abs(fftshift(fft(seq_sur(1,1:num_t_axis_plot))))))

xlabel('Frequency (MHz)')

ylabel('Amplitude (dB)')

axis([-f_s/2/1e6,f_s/2/1e6,-100,0])

title('Spectrum (raw)')

subplot(6,2,9)

plot(t_axis_plot*1e3,real(seq_sur_ddc(1,1:num_t_axis_plot)))

xlabel('Time (ms)')

ylabel('Amplitude')

axis([0,duration_plot*1e3,-2e-3,2e-3])

title('Surveillance signal (after DDC)')

subplot(6,2,10)

plot(f_axis_plot/1e6,20*log10(abs(fftshift(fft(seq_sur_ddc(1,1:num_t_axis_plot))))))

xlabel('Frequency (MHz)')

ylabel('Amplitude (dB)')

axis([-f_s/2/1e6,f_s/2/1e6,-100,0])

title('Spectrum (after DDC)')

subplot(6,2,11)

plot(t_axis_plot*1e3,real(seq_sur_lpf(1,1:num_t_axis_plot)))

xlabel('Time (ms)')

ylabel('Amplitude')

axis([0,duration_plot*1e3,-2e-3,2e-3])

title('Surveillance signal (after LPF)')

subplot(6,2,12)

plot(f_axis_plot/1e6,20*log10(abs(fftshift(fft(seq_sur_lpf(1,1:num_t_axis_plot))))))

xlabel('Frequency (MHz)')

```

```

        ylabel('Amplitude (dB)')

        axis([-f_s/2/1e6,f_s/2/1e6,-100,0])

        title('Spectrum (after LPF)')

% Ambiguity function

fprintf('[stat]   Ambiguity processing.\n')

% num_loop = length(array_sample_shift)*length(array_Doppler_frequency);

% t_axis = 0:1/f_s:(duration*f_s-1)/f_s;

% temp = zeros(1,num_loop);

% f_axis_plot = linspace(-f_s/2, f_s/2, length(seq_ref_lpf)); % FFT 频率轴

for idx_sample_shift = 1:length(array_sample_shift)

    % idx_f_d = 1:length(array_Doppler_frequency)

    % idx_sample_shift = ceil(idx_RD/length(array_Doppler_frequency));

    % idx_f_d = mod(idx_RD-1,length(array_Doppler_frequency))+1;

    sample_shift = array_sample_shift(idx_sample_shift);           %从数组[0:5]中取值

    % f_d = array_Doppler_frequency(idx_f_d);                       %从数组[-40:2:40]中取值

    fft_result = fftshift(fft(seq_sur_lpf(1,1+sample_shift:end).*conj(seq_ref_lpf(1,1:end-sample_shift))));

    f_axis_plot = linspace(-f_s/2, f_s/2, length(fft_result)); % 根据 FFT 长度动态生成频率轴

    A_TRD_fft(idx_start_time, idx_sample_shift,:) = interp1(f_axis_plot, fft_result,
array_Doppler_frequency, 'linear', 0);

    % A_TRD(idx_start_time, idx_sample_shift, idx_f_d) = abs(temp(1, idx_RD));

end

% Ambiguity function

fprintf('[stat]   Ambiguity processing.\n')

num_loop = length(array_sample_shift)*length(array_Doppler_frequency);

t_axis = 0:1/f_s:(duration*f_s-1)/f_s;

temp = zeros(1,num_loop);

```

```

for idx_RD = 1:num_loop

    idx_sample_shift = ceil(idx_RD/length(array_Doppler_frequency));

    idx_f_d = mod(idx_RD-1,length(array_Doppler_frequency))+1;

    sample_shift = array_sample_shift(idx_sample_shift);           %从数组[0:5]中取值

    f_d = array_Doppler_frequency(idx_f_d);                       %从数组[-40:2:40]中取值

    temp(1,idx_RD) = sum(seq_sur_lpf(1,1+sample_shift:end) ...

        .*conj(seq_ref_lpf(1,1:end-sample_shift)) ...

        .*exp(-1i*2*pi*f_d*t_axis(1+sample_shift:end)));

    A_TRD_dwt(idx_start_time, idx_sample_shift, idx_f_d) = abs(temp(1, idx_RD));

end

end

idx_max_range = zeros(1,length(array_start_time))-1;

idx_max_Doppler_frequency = zeros(1,length(array_start_time));

thres_A_TRD = -10;

for idx_start_time = 1:length(array_start_time)

    fig = figure(idx_start_time+1);

    ScreenSize = get(0,'ScreenSize');

    set(fig,'Position',[0.75*ScreenSize(3)+50,0.5*ScreenSize(4)+50,0.25*ScreenSize(3)-100,0.5*ScreenSize(4)-150]);

    [meshgrid_Doppler,meshgrid_range] = meshgrid(array_Doppler_frequency,[array_range,2*array_range(end)-array_range(end-1)]);

    plot_A_RD_fft = abs(squeeze(A_TRD_fft(idx_start_time,:,:)));

    plot_A_RD_fft = plot_A_RD_fft/max(max(plot_A_RD_fft));

    plot_A_RD_fft = 20*log10(plot_A_RD_fft);

    plot_A_RD_fft(plot_A_RD_fft<thres_A_TRD) = thres_A_TRD;

    plot_A_RD_fft = [plot_A_RD_fft;thres_A_TRD*ones(1,size(plot_A_RD_fft,2))];

    surf(meshgrid_Doppler,meshgrid_range,plot_A_RD_fft)

    view(0,90)

    colorbar

```

```

xlim([array_Doppler_frequency(1),array_Doppler_frequency(end)])

ylim([array_range(1),2*array_range(end)-array_range(end-1)])

xticks([array_Doppler_frequency(1):20:array_Doppler_frequency(end)])

yticks([array_range,2*array_range(end)-array_range(end-1)])

xlabel('Doppler frequency (Hz)')

ylabel('Range (m)')

[idx_max_range(idx_start_time),idx_max_Doppler_frequency(idx_start_time)] =
find(plot_A_RD_fft==max(max(plot_A_RD_fft)));

temp = sprintf('Range-Doppler Spectrum [%4.1fs: %3.0fm %3.0fHz]', ...

    array_start_time(idx_start_time), ...

    array_range(idx_max_range(idx_start_time)), ...

    array_Doppler_frequency(idx_max_Doppler_frequency(idx_start_time)));

title(temp)

end

% Plot TD Spectrum

fig101 = figure(101);

ScreenSize = get(0,'ScreenSize');

set(fig101,'Position',[0.5*ScreenSize(3)+50,50,0.25*ScreenSize(3)-100,0.5*ScreenSize(4)-150]);

[meshgrid_Doppler,meshgrid_start_time] = ...

    meshgrid(array_Doppler_frequency,[array_start_time,array_start_time(end)+duration]);

plot_A_TD_fft = zeros(length(array_start_time),length(array_Doppler_frequency));

for idx_start_time = 1:length(array_start_time)

    plot_A_TD_fft(idx_start_time,:) =
abs(squeeze(A_TRD_fft(idx_start_time,idx_max_range(idx_start_time),:)));

end

plot_A_TD_fft = plot_A_TD_fft./max(plot_A_TD_fft,[],2);

plot_A_TD_fft = 20*log10(plot_A_TD_fft);

plot_A_TD_fft(plot_A_TD_fft<thres_A_TRD) = thres_A_TRD;

plot_A_TD_fft = [plot_A_TD_fft;thres_A_TRD*ones(1,size(plot_A_TD_fft,2))];

surf(meshgrid_Doppler,meshgrid_start_time,plot_A_TD_fft)

view(0,90)

```

```

colorbar

xlim([array_Doppler_frequency(1),array_Doppler_frequency(end)])

ylim([array_start_time(1),array_start_time(end)])

xticks([array_Doppler_frequency(1):20:array_Doppler_frequency(end)])

yticks([array_start_time(1):0.5:array_start_time(end)+duration])

xlabel('Doppler frequency (Hz)')

ylabel('Time (s)')

title('Time-Doppler Spectrum')

idx_max_range = zeros(1,length(array_start_time))-1;

idx_max_Doppler_frequency = zeros(1,length(array_start_time));

thres_A_TRD = -10;

for idx_start_time = 1:length(array_start_time)

    fig = figure(idx_start_time);

    ScreenSize = get(0,'ScreenSize');

    set(fig,'Position',[0.75*ScreenSize(3)+50,0.5*ScreenSize(4)+50,0.25*ScreenSize(3)-
100,0.5*ScreenSize(4)-150]);

    [meshgrid_Doppler,meshgrid_range] =
meshgrid(array_Doppler_frequency,[array_range,2*array_range(end)-array_range(end-1)]);

    plot_A_RD_dwt = abs(squeeze(A_TRD_dwt(idx_start_time,:,:)));

    plot_A_RD_dwt = plot_A_RD_dwt/max(max(plot_A_RD_dwt));

    plot_A_RD_dwt = 20*log10(plot_A_RD_dwt);

    plot_A_RD_dwt(plot_A_RD_dwt<thres_A_TRD) = thres_A_TRD;

    plot_A_RD_dwt = [plot_A_RD_dwt;thres_A_TRD*ones(1,size(plot_A_RD_dwt,2))];

    surf(meshgrid_Doppler,meshgrid_range,plot_A_RD_dwt)

    view(0,90)

    colorbar

    xlim([array_Doppler_frequency(1),array_Doppler_frequency(end)])

    ylim([array_range(1),2*array_range(end)-array_range(end-1)])

    xticks([array_Doppler_frequency(1):20:array_Doppler_frequency(end)])

    yticks([array_range,2*array_range(end)-array_range(end-1)])

    xlabel('Doppler frequency (Hz)')

    ylabel('Range (m)')

```

```

        [idx_max_range(idx_start_time),idx_max_Doppler_frequency(idx_start_time)] =
        find(plot_A_RD_dwt==max(max(plot_A_RD_dwt)));

        temp = sprintf('Range-Doppler Spectrum [%4.1fs: %3.0fm %3.0fHz]', ...

            array_start_time(idx_start_time), ...

            array_range(idx_max_range(idx_start_time)), ...

            array_Doppler_frequency(idx_max_Doppler_frequency(idx_start_time)));

        title(temp)

end

% Plot TD Spectrum

fig102 = figure(102);

ScreenSize = get(0,'ScreenSize');

set(fig102,'Position',[0.5*ScreenSize(3)+50,50,0.25*ScreenSize(3)-100,0.5*ScreenSize(4)-150]);

[meshgrid_Doppler,meshgrid_start_time] = ...

    meshgrid(array_Doppler_frequency,[array_start_time,array_start_time(end)+duration]);

plot_A_TD_dwt = zeros(length(array_start_time),length(array_Doppler_frequency));

for idx_start_time = 1:length(array_start_time)

    plot_A_TD_dwt(idx_start_time,:) =
    abs(squeeze(A_TRD_dwt(idx_start_time,idx_max_range(idx_start_time),:)));

end

plot_A_TD_dwt = plot_A_TD_dwt./max(plot_A_TD_dwt,[],2);

plot_A_TD_dwt = 20*log10(plot_A_TD_dwt);

plot_A_TD_dwt(plot_A_TD_dwt<thres_A_TRD) = thres_A_TRD;

plot_A_TD_dwt = [plot_A_TD_dwt;thres_A_TRD*ones(1,size(plot_A_TD_dwt,2))];

surf(meshgrid_Doppler,meshgrid_start_time,plot_A_TD_dwt)

view(0,90)

colorbar

xlim([array_Doppler_frequency(1),array_Doppler_frequency(end)])

ylim([array_start_time(1),array_start_time(end)])

xticks([array_Doppler_frequency(1):20:array_Doppler_frequency(end)])

yticks([array_start_time(1):0.5:array_start_time(end)+duration])

xlabel('Doppler frequency (Hz)')

```

```

ylabel('Time (s)')

title('Time-Doppler Spectrum')


% Plot TD Spectrum

fig103 = figure(103);

ScreenSize = get(0,'ScreenSize');

set(fig103,'Position',[0.5*ScreenSize(3)+50,50,0.25*ScreenSize(3)-100,0.5*ScreenSize(4)-150]);

[meshgrid_Doppler,meshgrid_start_time] = ...

    meshgrid(array_Doppler_frequency,[array_start_time,array_start_time(end)+duration]);

plot_A_TD = zeros(length(array_start_time),length(array_Doppler_frequency));


plot_A_TD = plot_A_TD_fft-plot_A_TD_dwt;


surf(meshgrid_Doppler,meshgrid_start_time,plot_A_TD)

view(0,90)

colorbar

xlim([array_Doppler_frequency(1),array_Doppler_frequency(end)])

ylim([array_start_time(1),array_start_time(end)])

xticks([array_Doppler_frequency(1):20:array_Doppler_frequency(end)])

yticks([array_start_time(1):0.5:array_start_time(end)+duration])

xlabel('Doppler frequency (Hz)')

ylabel('Time (s)')

title('Time-Doppler Spectrum')

```